

Sri Lanka Institute of Information Technology



IT2130 – Operating Systems and System Administration
Year 2, Semester 2- 2026

Practical Test Answer Submission

Student ID	IT24104205
Student Name	Harischandra M.A.W.V
Campus/ Center name	Malabe
Specialization	ISE
Weekday/ Weekend	Weekend
Batch No	01.01

Question 1

```
#include <stdio.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>


int main() {

    int n;


    printf("Enter a positive integer: ");

    scanf("%d", &n);


    pid_t pid = fork();


    if (pid == 0) {

        // Child Process

        printf("\nChild Process\n");

        printf("PID: %d, PPID: %d\n", getpid(), getppid());


        int a = 0, b = 1, next;


        printf("Fibonacci series: ");
```

```
    for (int i = 0; i < n; i++) {  
        printf("%d ", a);  
        next = a + b;  
        a = b;  
        b = next;  
    }  
    printf("\n");  
}  
else {  
    // Parent Process  
    wait(NULL);  
    printf("\nParent: Child process completed.\n");  
}  
  
return 0;  
}
```

Question 2

```
#include <stdio.h>

#include <pthread.h>

#include <unistd.h>

void* findPrimes(void* arg) {

    int n = *(int*)arg;

    printf("\nWorker Thread\n");

    printf("Thread ID: %lu\n", pthread_self());

    printf("Process ID: %d\n", getpid());

    printf("Prime numbers up to %d: ", n);

    for (int i = 2; i <= n; i++) {

        int isPrime = 1;

        for (int j = 2; j <= i / 2; j++) {

            if (i % j == 0) {

                isPrime = 0;

                break;

            }

        }

    }

}
```

```
}
```

```
    if (isPrime)
```

```
        printf("%d ", i);
```

```
    }
```

```
    printf("\n");
```

```
    return NULL;
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    pthread_t thread;
```

```
    printf("Enter a positive integer: ");
```

```
    scanf("%d", &n);
```

```
    pthread_create(&thread, NULL, findPrimes, &n);
```

```
    pthread_join(thread, NULL);
```

```
    printf("\nMain Thread: Worker thread finished.\n");
```

```
    return 0;
```

```
}
```

Question 3

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
#define MAX 100
```

```
int arr[MAX];
```

```
int partialSum[MAX];
```

```
typedef struct {
```

```
    int start;
```

```
    int end;
```

```
    int index;
```

```
} ThreadData;
```

```
void* computeSum(void* arg) {
```

```
    ThreadData data = *(ThreadData*)arg;
```

```
    int sum = 0;
```

```
    for (int i = data.start; i <= data.end; i++) {  
        sum += arr[i];  
    }  
  
    partialSum[data.index] = sum;  
  
    printf("\nThread ID: %lu\n", pthread_self());  
    printf("Process ID: %d\n", getpid());  
    printf("Range: %d to %d\n", data.start, data.end);  
    printf("Partial Sum: %d\n", sum);  
  
    return NULL;  
}  
  
int main() {  
    int n, t;  
  
    printf("Enter size of array: ");  
    scanf("%d", &n);  
  
    printf("Enter %d elements:\n", n);  
    for (int i = 0; i < n; i++) {  
        scanf("%d", &arr[i]);
```

```
}
```

```
printf("Enter number of threads: ");
```

```
scanf("%d", &t);
```

```
pthread_t threads[t];
```

```
ThreadData data[t];
```

```
int chunk = n / t;
```

```
int start = 0;
```

```
for (int i = 0; i < t; i++) {
```

```
    data[i].start = start;
```

```
    data[i].end = (i == t - 1) ? n - 1 : start + chunk - 1;
```

```
    data[i].index = i;
```

```
    pthread_create(&threads[i], NULL, computeSum, &data[i]);
```

```
    start = data[i].end + 1;
```

```
}
```

```
int totalSum = 0;
```



```
for (int i = 0; i < t; i++) {  
    pthread_join(threads[i], NULL);  
    totalSum += partialSum[i];  
}  
  
float average = (float)totalSum / n;  
  
printf("\nTotal Sum: %d\n", totalSum);  
printf("Average: %.2f\n", average);  
  
return 0;  
}
```